“FormSync” - Cross-Platform Form Generator with Schema-Driven Automation

Thamindu Dissanayake
Faculty of Computing
Sri Lanka Institute of Information Technology
Malabe, Sri Lanka
it22003546@my.sliit.lk

Thihansi Gunawardena
Faculty of Computing
Sri Lanka Institute of Information Technology
Malabe, Sri Lanka
it22350114@my.sliit.lk

Yasas Rajapakshe
Faculty of Computing
Sri Lanka Institute of Information Technology
Malabe, Sri Lanka
it22305350@my.sliit.lk

Movindu Liyanage
Faculty of Computing
Sri Lanka Institute of Information Technology
Malabe, Sri Lanka
it22332608@my.sliit.lk

Jeewaka Perera
Faculty of Computing
Sri Lanka Institute of Information Technology
Malabe, Sri Lanka
jeewaka.p@sliit.lk

Thilini Jayalath
Faculty of Computing
Sri Lanka Institute of Information Technology
Malabe, Sri Lanka
thilini.j@sliit.lk

***Abstract*— Building form-based web applications requires re-implementing identical validation constraints across frontend, backend, and testing layers, leading to redundancy and inconsistencies as requirements evolve. Existing tools address these concerns in isolation, without providing a unified mechanism for cross-layer validation consistency. This paper presents FormSync, a schema-driven platform that uses JSON Schema as a single source of truth for full-stack artifact generation. From a single validated schema, FormSync generates a frontend form interface, a backend service with CRUD functionality, and a constraint-driven test suite, ensuring consistent validation across all layers without manual intervention. The platform additionally includes an LLM-assisted schema authoring pipeline, multi-format input support, and a composite schema quality scoring mechanism. Evaluation across schemas of 5 to 40 fields showed end-to-end generation latency under two seconds, with caching reducing latency by up to 94%. The LLM enhancement pipeline achieved a 94.2% valid suggestion rate. Comparative analysis indicates that FormSync enables complete cross-layer validated artifact generation, including automated test synthesis, from a single schema definition.**

Keywords*— *Schema-driven code generation, Cross-layer validation synchronization, Full-stack automation, Model-driven engineering, LLM-assisted schema authoring, Constraint-driven test synthesis, JSON Schema.

I. Introduction

Forms are among the most fundamental interaction primitives in modern web applications. They support user registration, data submission, configuration workflows, and transactional operations across nearly every application domain. Despite this central role, the engineering process for building form-based systems remains fragmented, repetitive, and inconsistent.

Prior software engineering studies have identified three recurring challenges in full-stack application development. Cross-layer validation inconsistency arises when identical constraints must be independently implemented in the frontend, backend, and persistence layer representations that diverge as requirements evolve, violating the Do not Repeat Yourself (DRY) principle [1] and introducing latent defects [2]. Scaffolding overhead describes the manual creation of entities, repositories, services, controllers, and test suites for each domain entity, consuming significant developer effort [3] without producing intellectual value, a cost well-discussed in model-driven engineering literature [4]. Schema co-evolution further compounds these challenges. When a data contract changes, the update must be applied consistently across the frontend, backend, and persistence layers. Maintaining this consistency typically requires coordinated manual updates, which is error-prone and poorly supported by existing development tools [5]. These problems are further aggravated by the exclusion of non-technical stakeholders from schema authoring, despite their domain expertise needed to define the data structures used by the application. Together, they represent a systematic gap between schema definition and full-stack implementation.

Existing tools address portions of this problem in isolation. Formik and React Hook Form manage frontend form state efficiently but produce no reusable schema artifact and provide no synchronization with backend validation. JSONForms generates User Interfaces from JavaScript Object Notation (JSON) Schema but does not synchronize with backend services and require schemas to be authored manually. JHipster and OpenAPI Generator produce backend scaffolding but rely on proprietary domain-specific languages or complete OpenAPI specifications as input, and neither integrates frontend form generation or accessibility-aware output. No existing system combines schema-driven frontend generation, synchronized backend service generation, automated test synthesis, LLM-assisted schema authoring, and persistent schema management within a unified workflow.

This paper presents FormSync, a platform designed around the principle that JSON Schema Draft-07 can serve as the single source of truth for an entire application layer. From this schema artifact, FormSync generates a React frontend application with accessible form components, a Spring Boot backend with full CRUD infrastructure, a JUnit 5 test suite with constraint-driven test data, while ensuring consistent validation across both layers. Furthermore, the platform integrates an LLM-powered schema authoring environment with multi-format parsing support, a quantitative schema quality engine, and a persistent template repository enabling iterative schema management across development sessions.

II. Related Work

A. Form State Management Libraries

Formik [2] and React Hook Form [4] are the dominant form management libraries in the React ecosystem, offering declarative field registration, submission handling, and validation. Both are mature and well-supported but operate exclusively at the frontend presentation layer. Neither produces a reusable schema artifact, neither is aware of backend validation constraints, and neither generates any server-side code.

B. Schema-Driven Form Rendering

JSONForms [5] is the closest prior work in schema-driven form generation. It renders React or Angular form components from a JSON Schema input combined with a UI schema for layout configuration and supports a visual editor for layout design. However, its scope is limited to frontend rendering: it does not generate backend code, does not synchronize validation with any server-side framework, and does not produce downloadable application artifacts. It further requires schemas to be manually authored in JSON, with no support for YAML or XML. Alpaca Forms and JSON Editor share this same limitation. [6] proposed a visual drag-and-drop form generator that renders across Vue, React, and Angular, but is equally confined to frontend rendering with no backend synchronization or accessibility compliance.

C. Backend Code Generation

OpenAPI Generator [7] and Swagger Codegen accept OpenAPI 3.x specifications and produce server stubs and client SDKs in multiple languages. These tools are valuable for teams that already possess a complete OpenAPI specification but generate skeleton code requiring manual completion of validation annotations [8], business logic, and test suites, and do not generate frontend form components. JHipster [9] provides end-to-end scaffolding from JDL, a proprietary DSL for entity modeling, generating a complete Spring Boot backend and Angular or React frontend. It is comprehensive but does not support JSON Schema as input, does not integrate AI-assisted authoring, and does not synthesize constraint-driven test suites.

D. Automated Interface Generation

pix2code [10], Prototype2Code [11], and Diffusion UI [12] demonstrated automated generation of interface code from screenshots and design prototypes using neural and diffusion-based models. [6] further proposed a CNN pipeline classifying mobile UI components to Android and iOS templates. These contributions establish the feasibility of automated interface generation, but all operate on visual inputs rather than declarative schemas, and none produce validation logic, backend artifacts, or synchronized multi-layer outputs.

E. Model-Driven Engineering

Model-Driven Engineering (MDE) promotes the use of abstract models as primary software artifacts, with code generated through model-to-text transformations [13]. Traditional tools; Eclipse Modeling Framework and Acceleo operate on UML or Ecore metamodels and require specialized modeling expertise. [14] demonstrated a UML-driven Acceleo generator producing synchronized frontend and backend artifacts across Angular, ReactJS, and React Native, but is limited to manual UML modeling and CRUD-level scaffolding with no JSON Schema or AI-assisted authoring support. FormSync applies MDE principles using JSON Schema, a format already familiar to web developers as the platform-independent model, lowering the barrier to entry while retaining cross-layer consistency benefits.

F. LLM-Assisted Schema Authoring

Zhang et al. demonstrated that LLMs can generate schemas for knowledge graphs from structured descriptions [15]. Mior showed that transformer-based models can infer JSON Schema constraints from example data, including required fields, enum values, and type constraints [16]. Neubauer demonstrated that LLM-based post-processing can improve schema quality and developer usability through description and constraint annotation [17]. These findings motivate the AI enhancement pipeline in FormSync, where a structured prompting strategy produces incremental, categorized improvements rather than wholesale schema regeneration.

III. Research Gap

The reviewed literature reveals a consistent pattern: existing tools address individual layers of form-based application development in isolation, with no system providing an integrated solution across all layers. Table I. summarizes the feature coverage of representative tools compared to the proposed FormSync platform.

TABLE I. FEATURE COMPARISON OF EXISTING TOOLS VS. PROPOSED FRAMEWORK

Feature	Form Libraries	JSON Forms	JHipster	OpenAPI Generator	Form Sync
Frontend Form Generation	✓	✓	Partial	✗	✓
Backend API Generation	✗	✗	✓	✓	✓
Automated Test Generation	✗	✗	✗	✗	✓
Cross-layer Validation	✗	Partial	Partial	Partial	✓
LLM-assisted Enhancement	✗	✗	✗	✗	✓
Multi-format Input	✗	✗	✗	JSON/YAML	✓(JSON/YAML/XML)
WCAG Accessibility	✗	✗	✗	✗	✓
Full-stack Code Export	✗	✗	✓	✓	✓
End-to-End Pipeline	✗	✗	Partial	Partial	✓

Three specific gaps motivate this research. First, no existing tool provides structural validation synchronization, the guarantee that frontend and backend enforce identical constraints derived from a single artifact, as a built-in property of the generation pipeline rather than a developer discipline. Second, constraint-driven automated test synthesis, where test data is generated directly from schema constraints rather than authored by hand, has not been applied in a full-stack, schema-driven generation context. Third, intelligent schema authoring support for non-technical stakeholders, combined with multi-format schema interoperability, has not been integrated into a form generation platform.

IV. Architecture of the Framework

A. Architectural Overview

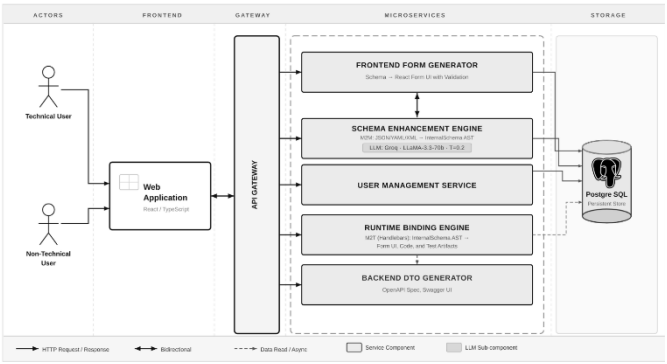

Fig. 1. Architecture of the Framework

The FormSync framework is built upon a decentralized microservices architecture, as depicted in Fig. 1, enabling high modularity, independent scalability, and strict adherence to the Single Source of Truth (SSoT) principle. By decomposing the system into functional domains, the architecture ensures that schema evolution, code generation logic, and user management can scale and fail independently. The system is organized as a monolithic repository containing six specialized microservices coordinated by a central API Gateway.

B. Microservices Pipeline

Each microservice addresses a specific responsibility within the pipeline. The Intelligent Schema Definition & AI Engine manages the dual-path authoring environment and integrates the Groq API [18] (LLaMA-3.3-70B) for real-time schema enhancement and multi-format parsing across JSON, YAML, and XML inputs. The Schema Management Service, implemented in NestJS, functions as the central repository for validated JSON Schema Draft-07 artifacts, providing versioning, metadata persistence, and quality metric endpoints. The Frontend Form Builder Service consumes the validated schema to produce a WCAG 2.1-compliant React application through a Vite-based export pipeline. The Backend DTO & Entity Generator transforms the same schema into a deployable Spring Boot MVC project, automating the synthesis of JPA entities, DTOs, and REST controllers. The Runtime Binding & Test Suite Generator derives JUnit 5 test assertions directly from schema validation constraints, ensuring specification-code synchronization without manual test authoring. Finally, the User Management & Authentication Service handles identity, role-based access control, and session persistence via Prisma ORM backed by PostgreSQL.

All inter-service communication is mediated by the API Gateway, which enforces JWT-based authentication, request routing, and rate limiting as cross-cutting concerns. Data consistency is maintained through a dual-persistence strategy: transactional user and project data is managed relationally through Prisma, while the JSON Schema artifact itself is versioned and propagated as the immutable SSoT from which all generated components are derived.

V. Methodology and Design

A. Schema Definition and Multi-format Parsing

The schema authoring subsystem provides two entry paths that converge into the same validated JSON Schema pipeline. The Technical Path provides a full-featured Monaco code editor with syntax highlighting for JSON, YAML, and XML, with real-time format detection, structural schema tree inspection, and immediate validation feedback using the AJV validator. An undo/redo history mechanism preserves each editing state with a timestamp and action label. An optional automated workflow mode provides format conversion, AI enhancement, and quality score recalculation without manual intervention between stages, enabling a more efficient schema development workflow.

The Non-Technical Path provides a template-driven visual builder where business analysts and product managers can construct schemas through palette of typed field templates including string, numeric, Boolean, email, date, and URL variants. It further offers a set of pre-constructed schema templates covering common use cases; contact, registration, booking, and job application forms Each field supports configurable constraints, including length limits, pattern, format, numeric range, and default value. The constructed schema definition is serialized and transferred to the technical editor through a single action, ensuring both paths produces the same standard JSON Schema representation.

A plugin-based parser architecture handles multi-format input. Three dedicated parsers are registered at application startup: a JsonParserPlugin that normalizes JSON input and preserves Draft-07 meta-schema references; a YamlParserPlugin that uses js-yaml plugin to parse YAML anchors and multi-line strings into standard JSON Schema objects; and an XmlParserPlugin that uses fast-xml-parser with attribute normalization to map XML element cardinality (minOccurs, maxOccurs) to JSON Schema array constraints. All three parsers emit the same standard JSON Schema Draft-07 object, which is the single source of truth json schema uses for further processes.

B. AI Assisted Schema Enhancement

LLM- powered enhancement requests are dispatched to the Groq-hosted LLaMA-3.3-70B-versatile model using an OpenAI-compatible endpoint, with the inference temperature fixed at T=0.2 to produce near-deterministic outputs across successive enhancement cycles. Prompt engineering safeguards mitigate non-determinism and hallucination through a three-layer validation pipeline. First, the system prompt enforces a strict output contract requiring the model to return the unchanged input schema and generate a structured suggestions array formatted as valid JSON, responses that deviate from this output format are rejected before reaching the client. Second, the prompt explicitly prohibits overwriting user-defined constraints including patterns, length bounds, and enumeration values; only absent properties may be proposed, thereby enforcing a human-in-the-loop review workflow. Third, every suggested mutation is validated against the JSON Schema Draft-07 specification using AJV [19]. Modifications that violate the specification (e.g., minimum greater than maximum, unknown keywords, or malformed $ref values) are automatically rejected by the Schema-API and discarded at the API boundary, ensuring that only specification-compliant proposals reach the user interface. Each suggestion contains a JSON Pointer target path, a quality category (validation, metadata, accessibility, or structure), the proposed schema modification and a natural-language explanation, and an estimated score impact. A client-side store filters duplicate proposals by comparing the target path and serialized rule content with previously accepted suggestions preventing re-proposal across successive enhancement cycles. When prior suggestions have

been applied, a quality recalculation request is issued with the full applied-suggestion context so that the reported score reflects cumulative human-validated improvements.

Schema quality is quantified by a composite score Q defined as:

$$Q = 0.25Sc + 0.25Vc + 0.20\,Ac + 0.20\,Cs + 0.10Ei \quad (1)$$

Equation (1), where Sc = structural completeness, Vc = , constraint coverage, Ac = accessibility metadata coverage, Cs = type-boundary consistency, and Ei = cumulative enhancement impact respectively. Each dimension is normalized to [0,1] before weighting, informed by the ISO/IEC 25010 quality framework [20]. The two highest-weighted dimensions, Sc and Vc, together contribute 50% of Q, reflecting the primacy of structural organization and validation constraints in machine-readable schemas. Q is recalculated after each accepted suggestion, providing continuous authoring feedback.

C. Frontend Form Builder

Within the FormSync architecture, the frontend generation mechanism converts the normalized schema representation into an interactive user interface. After the schema parsing and normalization stages, the system produces an intermediate model that describes form fields, layout structure, styling properties, and validation rules. This representation allows the system to generate user interfaces without directly depending on the syntactic details of the original JSON Schema.

Using this intermediate model, the system constructs the form interface by mapping each field definition to an appropriate input element. Field definitions within the schema are mapped to corresponding user interface elements, including text inputs, selection lists, boolean controls, and numeric inputs. Layout metadata defined in the schema determines how these elements are organized in the generated interface. Simple forms are rendered as sequential field groups, while more complex configurations including multi-step forms are divided into separate sections. Nested schema structures are handled recursively, allowing grouped inputs and hierarchical data structures to be represented in the interface.

Validation rules defined in the schema are incorporated into the generated frontend interface. Constraints including required fields, value ranges, and pattern restrictions are translated into corresponding client-side validation checks. Upon form submission, these constraints are evaluated and structured feedback is generated to identify invalid inputs and present appropriate error messages.

Accessibility considerations are embedded into the generated interface. Input elements are automatically associated with their labels, descriptive messages are connected using Accessible Rich Internet Applications (ARIA) [21] attributes, and submission errors are communicated through screen-reader-friendly announcements.

Because the frontend interface and backend validation logic are both derived from the same schema definition, FormSync ensures that validation rules remain consistent across the application layers. This prevents discrepancies between client-side and server-side validation and helps maintain reliable enforcement of input constraints.

D. Backend Service Generator

The backend generation subsystem transforms a JSON Schema into a Spring Boot 3.2 project. The generation pipeline proceeds through eight steps: schema normalization, schema-to-internal-model mapping via SchemaMapper, Maven Project-Object-Model (POM) generation, Java enum class generation, Spring Boot application class generation, per-entity MVC stack generation, documentation generation, and ZIP packaging.

The SchemaMapper transforms JSON Schema into a language-agnostic Internal Schema composed of SchemaEntity and SchemaField objects. Nine core DataType constants, shown in Table II, abstract over both JSON Schema types and Java-specific types. The isRoot flag on each entity controls generation scope: root entities receive the full six-layer stack (Entity, Request DTO, Response DTO, Repository, Service, Controller), while nested object entities are generated as @Embeddable Java classes without REST infrastructure

TABLE II. DATATYPE MAPPING JSON TO JAVA

DataType	JSON Schema Source	Java Target
STRING	type: "string"	String
INTEGER	type: "integer"	Integer
DOUBLE	type: "number"	Double
BOOLEAN	type: "boolean"	Boolean
LOCAL_DATE	format: "date"	java.time.LocalDate
LOCAL_DATE_TIME	format: "date-time"	java.time.LocalDateTime
ENUM	type:"string"+enum array	Java enum class
LIST	type: "array"	List<T>
OBJECT	type:"object"+ properties	Custom entity class

Code generation uses a Handlebars template engine with custom helpers that encapsulate Java-specific concerns: type mapping (toJavaType), naming convention (toSnakeCase, toKebabCase), regex literal escaping (escapeJava), and test data synthesis (testValue, belowMin, aboveMax). The escapeJava helper ensures that regex patterns from JSON Schema have correctly double escaped for Java string literals. The toJavaType helper returns SafeString instances to prevent HTML-encoding of angle brackets in parameterized type references. The generated project includes springdoc-openapi integration, which exposes a Swagger UI and machine-readable OpenAPI specification at runtime. Generated entity and DTO classes use Lombok annotations to reduce accessor boilerplate.

E. Test Suite Generator

The test synthesis subsystem generates a three-layer JUnit 5 test suite for each root entity. The ControllerTest class uses @SpringBootTest with @AutoConfigureMockMvc to run HTTP integration tests through a MockMvc context. The ServiceTest class uses @ExtendWith(MockitoExtension.class) for unit testing with mocked repository dependencies. The RepositoryTest class uses @DataJpaTest to verify persistence operations against an embedded H2 database.

The central design contribution in this subsystem is constraint-driven test data synthesis. The synthesis algorithm operates at the level of individual field constraints, treating each field as an independent unit whose valid and invalid test values are derived solely from its own JSON Schema

metadata. For each field in the schema, the system generates a valid test value for positive test cases and, for each declared constraint, an invalid boundary value for negative test cases. The testValue helper synthesizes valid data from constraint metadata: numeric fields use the midpoint of the declared range; string fields with a pattern constraint receive a heuristically generated matching example; email fields receive a well-formed address; enum fields receive the first declared constant. The belowMin, aboveMax, tooShortValue, and tooLongValue helpers compute boundary-violating values for negative test cases, each expected to elicit an HTTP 400 Bad Request response from the generated controller.

The pattern-aware example generator applies recognition heuristics for common regex families, including UUID patterns, phone number patterns, prefix-identifier patterns (for example, ^EVT-\d{3}$), digit-only strings, alphanumeric strings, IP addresses, and hex color codes. For patterns outside these families, a generic fallback value is substituted. This approach trades completeness for implementation practicality; the heuristic set represents the most frequently encountered patterns in web API schemas.

It is important to note that the current synthesis algorithm does not address cross-field relational constraints, that is, constraints whose validity depend on the relationship between two or more fields, for example, requiring that endDate be strictly greater than startDate, or that a confirmPassword field match a password field. Such constraints cannot be expressed as first-class constructs within JSON Schema Draft-07 without resorting to custom extension keywords, and their test data synthesis would require a constraint-solving strategy beyond the scope of the present work. This limitation is discussed further in Section VII (Limitations).

VI. Results and Evaluation

A. Response Time and Generation Performance

End-to-end latency was measured over 30 repeated requests per schema size (5, 10, 20, and 40 fields) on a local development environment. The host machine was equipped with an AMD Ryzen 7 processor and 16 GB of RAM, running Windows as the underlying operating system, with median values reported. Frontend render latency ranged from 312 ms to 1,394 ms, while backend ZIP generation ranged from 428 ms to 1,872 ms, both increasing with field count and exhibiting a near-linear trend. Redis cache hits reduced latency by 89–94% compared to cold requests. All uncached responses completed within two seconds, consistent with established interactive response thresholds for developer tooling

B. LLM Model Comparison and Selection

Four LLM configurations were evaluated on validity rate, latency, format compliance, and cost (Table III). LLaMA-3.3-70B on Groq [18] was selected, achieving the highest valid suggestion rate (94.2%) and format compliance (97.8%) at 610 ms p95 latency and no direct token cost in the evaluated configuration. The smaller LLaMA-3.1-8B variant produced malformed output in ~15% of requests and was excluded.

TABLE III. LLM MODEL COMPARISON

Model	Valid Rate (%)	Latency (ms)	Format Comp. (%)	Cost /1K tok.
LLaMA-3.3-70B (Groq)	94.2	610	97.8	$0.002
GPT-4o-mini (OpenAI)	91.7	1,240	95.1	$0.15
Gemini 1.5 Flash (Google)	89.3	980	93.4	$0.075
LLaMA-3.1-8B (Groq)	76.1	390	84.6	$0.001

C. Edge Case Handling

Six edge case categories were tested: empty schemas, deeply nested objects (up to five levels), conflicting constraints, unknown keywords, single-constant enums, and mixed required/optional fields. All six were handled correctly conflicting constraints were rejected at the AJV boundary before reaching generation, unknown keywords were silently ignored per Draft-07, and nested entities beyond the isRoot boundary were emitted as @Embeddable classes without REST infrastructure.

D. Comparative Analysis

FormSync was compared against JSONForms, JHipster, and OpenAPI Generator using a representative 15-field registration schema (Table IV). FormSync was observed to be the only evaluated tool to deliver a complete, compliable full-stack artifact with synchronized cross-layer validation and a constraint-driven test suite-in under five minutes. Competing tools required significantly higher and left validation logic, business logic, or persistence configuration to be completed manually.

TABLE IV. COMPARATIVE STUDY: FORMSYNC VS. EXISTING TOOLS (15-FIELD SCHEMA)

Tool	Full-Stack	Validation Sync	Tests Generated	Dev. Effort
FormSync	Yes	Yes	Yes	Low
JSONForms	No	No	No	High
JHipster	Yes	Partial	No	Medium
OpenAPI Generator	Partial	Partial	No	Very High

As illustrated in (Table IV), FormSync achieves its primary design objective, eliminating manual cross-layer alignment from schema-driven full-stack development while maintaining practical response times and robust edge-case resilience.

VII. Limitations

Several limitations of the proposed system are noted. The schema enhancement pipeline depends on an external large language model, introducing non-deterministic behavior that may affect reproducibility of generated outputs across executions. The schema quality scoring mechanism is based on heuristic weight assignments derived from iterative design decisions, which have not yet been validated against practitioner-defined quality criteria. These limitations may be mitigated in future work through deterministic sampling configurations and empirical calibration of scoring weights.

The system was evaluated using synthetic schemas constructed to cover targeted field types and constraint categories. Further validation using large-scale, real-world schemas and formal user studies is required to fully assess

robustness and usability in practical deployment scenarios. The generated artifacts are currently limited to React for frontend development and Spring Boot for backend services. Although the generation pipeline is designed with extensibility in mind, support for alternative frameworks, as examples, Angular, Vue, and Django has not yet been implemented, which limits applicability for teams using different technology stacks.

VIII. CONCLUSION

This paper presented FormSync, a schema-driven platform that addresses fragmentation in form-based web application development by generating a consistent set of full-stack artifacts from a single JSON Schema source of truth. From one schema definition, FormSync produces a WCAG 2.1-compliant React frontend, a Spring Boot backend with complete CRUD infrastructure, a constraint-driven JUnit 5 test suite, and synchronized cross-layer validation, eliminating the manual alignment required by all current approaches.

Five contributions were realized: structural validation synchronization across frontend, backend, and test layers; constraint-driven boundary-value test data synthesis from schema metadata; a provider-agnostic LLM enhancement pipeline with a quantitative quality scoring engine grounded in ISO/IEC 25010; a multi-format plugin-based parser supporting JSON, YAML, XML and a persistent schema template repository enabling reuse and versioning.

Evaluation demonstrated that FormSync produces compilable, runnable artifacts across schemas of 5 to 40 fields with sub-two-second generation latency, and outperformed JSONForms, JHipster, and OpenAPI Generator in end-to-end delivery time and cross-layer completeness. Future work will extend the backend generator to support inter-entity relationships via $ref resolution, conduct formal usability and accessibility evaluation, and assess fine-tuned LLM variants for schema enhancement quality.

REFERENCES

[1] A. Hunt and D. Thomas, The Pragmatic Programmer, Boston, MA, USA: Addison-Wesley, 1999.

[2] F. Team, "Formik: Build forms in React, without tears," 2023. [Online]. Available: https://formik.org/docs/overview. [Accessed 14 Aug 2025].

[3] K. Czarnecki and U. W. Eisenecker, Generative Programming: Methods, Tools, and Applications, Boston, MA, USA: Addison-Wesley, 2000.

[4] R. H. F. Contributors, "“React Hook Form Documentation”," 2025. [Online]. Available: https://react-hook-form.com.

[5] J. Team, "JSON Forms Documentation," JSONForms, [Online]. Available: https://jsonforms.io/docs/. [Accessed 12 Sep 2025].

[6] S. Chen, L. Fan, T. Su, L. Ma, Y. Liu and L. Xu, "Automated cross-platform GUI code generation for mobile apps," in *Proc. 1st IEEE International Workshop on Artificial Intelligence for Mobile*, Hangzhou, China, 2019.

[7] S. Software, "OpenAPI Generator," OpenAPI Generator, [Online]. Available: https://openapi-generator.tech. [Accessed 10 Oct 2025].

[8] V. Samotyy, U. Dzelendzyak and N. Mashtaler, "A comparative study of data annotations and fluent validation in .NET," *International Journal of Computing,* vol. 23, no. 1, p. doi: 10.47839/ijc.23.1.3437, 2024.

[9] J. Project, "JHipster - Full Stack Platform for the Modern Developer," JHipster, [Online]. Available: https://www.jhipster.tech. [Accessed 09 Oct 2025].

[10] T. Beltramelli, "pix2code: Generating code from a graphical user interface screenshot," arXiv preprint arXiv:1705.07962, 2017.

[11] S. Xiao, Y. Chen, J. Li, L. Sun and T. Zhou, "Prototype2Code: End-to-end frontend code generation from UI design prototypes," in *Proc. ASME IDETC/CIE*, Washington, DC, USA, 2024.

[12] Y. Duan, L. Yang, T. Zhang, Z. Song and F. Shao, "Automated UI interface generation via diffusion models," in *Proc. Int. Symp. Computer Applications and Information Technology (ISCAIT)*, Xi’an, China, 2025.

[13] D. C. Schmidt, "Model-driven engineering," *IEEE Computer,* vol. 39, no. 2, p. 25–31, 2006.

[14] M. Inayatullah, F. Azam and M. W. Anwar, "Model-based scaffolding code generation for cross-platform applications," in *Proc. IEEE 10th Annual Information Technology, Electronics and Mobile Communication Conference (IEMCON)*, Islamabad, Pakistan, 2019.

[15] B. Zhang, Y. He, L. Pintscher, A. M. Penuela and E. Simperl, "Schema generation for large knowledge graphs using large language models," arXiv preprint arXiv:2501.08608, 2025.

[16] M. J. Mior, "Large language models for JSON schema discovery," arXiv preprint arXiv:2407.03286, 2024.

[17] J. P. B. U. F. Neubauer, "AI-assisted JSON schema creation and mapping," arXiv preprint arXiv:2508.05192, 2024.

[18] G. Inc., "Groq LPU Inference Engine and Performance Metrics," Groq Inc. [Online]. [Accessed 19 Sep 2025].

[19] E. Zaidman and Contributors, "Ajv: Another JSON Schema Validator," 2024. [Online]. Available: https://ajv.js.org/. [Accessed 18 Sep 2025].

[20] ISO/IEC, "Systems and software engineering - Systems and software quality requirements and evaluation (SQuaRE) - Product quality model," in *ISO/IEC 25010:2011*, 2011.

[21] W3C, "Accessible Rich Internet Applications (WAI-ARIA) 1.2," W3C, [Online]. Available: https://www.w3.org/TR/wai-aria-1.2/. [Accessed 13 Aug 2025].